React: un'applicazione completa

Corso di Frontend Development – Modulo 01

Federico Giuggioloni

ITS – Corso di Frontend Development

Anno formativo 2026/2027

Contenuto del modulo

1. React: un'applicazione completa
1.1 Da dove partiamo
1.2 Ripasso di React sul codice del progetto
1.3 Routing
1.4 Autenticazione
1.5 I dati dal server
1.6 Form e validazione
1.7 Stato dell'applicazione
1.8 Interfaccia: Tailwind e shadcn/ui
1.9 Funzionalità da prodotto vero
1.10 Robustezza e prestazioni
1.11 Build e pubblicazione

Sommario del modulo

1. React: un'applicazione completa
1.1 Da dove partiamo
1.2 Ripasso di React sul codice del progetto
1.3 Routing
1.4 Autenticazione
1.5 I dati dal server
1.6 Form e validazione
1.7 Stato dell'applicazione
1.8 Interfaccia: Tailwind e shadcn/ui
1.9 Funzionalità da prodotto vero
1.10 Robustezza e prestazioni
1.11 Build e pubblicazione

Obiettivo del modulo

- minicommerce *funziona*. Non è ancora un **prodotto**.
- Differenza fra le due cose: cache, URL condivisibili, errori gestiti in pagina, form robusti, prestazioni, build pubblicata.
- In questo modulo tocchiamo quasi solo `frontend/src`: il backend resta com'è.
- Metodo: per ogni argomento guardiamo **il codice che c'è**, diciamo cosa non va, e lo miglioriamo.

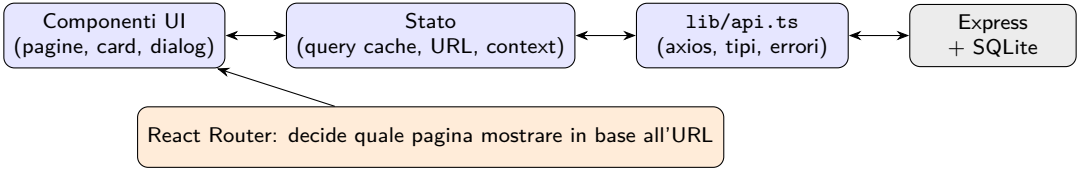
Cosa aggiungiamo allo stack

Un hook per i dati scritto da noi, react-hook-form + zod (form), filtri nell'URL con React Router, Error Boundary, code splitting.

La lista della spesa del modulo

Com'è oggi	Dove lo portiamo
`useEffect` + `useState` copiati in ogni pagina	un hook `useProducts` solo, con annullamento e *reload*
Filtri in `useState`	filtri nell'URL: link condivisibili, tasto Indietro
Errori solo in un toast che sparisce	stato di errore in pagina con pulsante Riprova
`/admin` e `/cart` si proteggono da sole	route protette da un layout, con il ruolo
401 non gestito	interceptor di risposta: sessione scaduta $\rightarrow$ login
Validazione del form scritta a mano	schema zod riusabile, meno codice
`window.confirm` per eliminare	dialog accessibile e coerente con il resto
Tutto il JS in un solo file	una route, un file: code splitting

Architettura di una SPA



- **Single Page Application**: un solo `index.html`, il JavaScript disegna tutto e parla con l'API in JSON.
- Il browser non ricarica mai la pagina: la navigazione è gestita dal router.
- Conseguenza: in produzione il server deve rispondere `index.html` a *qualsiasi* percorso. Ci torniamo a fine modulo.

Componenti e props

```tsx
// una card del catalogo, estratta da CatalogPage in un componente suo
interface ProductCardProps {
  product: Product;
  onEdit?: (product: Product) => void;    // opzionale: nel catalogo non serve
}

export function ProductCard({ product, onEdit }: ProductCardProps) {
  return (
    <Card className="flex flex-col">
      <CardTitle className="text-base">{product.name}</CardTitle>
      <Badge variant="secondary">{product.category}</Badge>
      <span className="text-lg font-bold">{formatPrice(product.price)}</span>
      {product.stock > 0
        ? <Badge variant="outline">Disponibili: {product.stock}</Badge>
        : <Badge variant="destructive">Esaurito</Badge>}
    </Card>
  );
}
```

Un componente è una **funzione pura**: stesse props, stesso risultato. I tipi delle props sono la sua documentazione.

State ed eventi: la ricerca del catalogo

```
const [search, setSearch] = useState('');

<Input
  placeholder="Cerca per nome, marca o descrizione..."
  value={search}
  onChange={(e) => setSearch(e.target.value)}
/>
```

- Un input con `value` + `onChange` è **controllato**: React è la fonte di verità, non il DOM.
- Lo stato vive nel componente più in basso che ne ha bisogno. Se serve a due fratelli, sale nel padre (*lifting*).
- In `CatalogPage` ci sono sei stati di filtro (`search`, `category`, `minPrice`, `maxPrice`, `sort`, `order`): fra poco li spostiamo tutti nell'URL, e il componente ne esce più semplice.

Liste, chiavi, rendering condizionale

```
{loading ? (
  <p className="py-12 text-center text-muted-foreground">Caricamento...</p>
) : products.length === 0 ? (
  <div className="flex flex-col items-center gap-2 py-12">
    <PackageX className="size-8" />
    <p>Nessun articolo trovato con i filtri selezionati.</p>
  </div>
) : (
  <div className="grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-3">
    {products.map((product) => <ProductCard key={product.id} product={product} />)}
  </div>
)}
```

- La `key` serve a React per capire *quale* elemento è cambiato: stabile e unica. Qui è `product.id`: corretto.
- Errore classico: `key={index}` con una lista che si riordina. Il catalogo si riordina a ogni cambio di `sort`.
- Il ternario annidato regge tre casi. Ne mancano due: **errore** e **aggiornamento in corso**. Li aggiungiamo nel modulo.

useEffect: quando sì e quando no

```
// Nel progetto: carica le categorie una volta sola, al montaggio
useEffect(() => {
  fetchCategories().then(setCategories).catch(err => toast.error(getErrorMessage(err)));
}, []);

// Nel progetto: ricarica gli articoli quando cambia un filtro, con debounce
useEffect(() => {
  const timer = setTimeout(() => { /* fetchProducts(...) */ }, 250);
  return () => clearTimeout(timer);     // pulizia: annulla il timer precedente
}, [search, category, minPrice, maxPrice, sort, order]);
```

- La funzione restituita è la **pulizia**: qui annulla il timer, ed è ciò che rende il debounce corretto.
- **Sì** agli effetti per sincronizzare con qualcosa fuori da React (timer, titolo della pagina, sottoscrizioni).
- **No** per calcolare dati derivati, e (fra poco vedremo perché) **no** per caricare dati dal server.

React 19: cosa usa già il progetto

```
// 1. Il Context si usa direttamente come provider: niente <AuthContext.Provider>
return <AuthContext value={{ user, login, logout }}>{children}</AuthContext>;

// 2. startTransition: logout e navigazione insieme, senza render intermedi
const handleLogout = () => {
  startTransition(() => { logout(); navigate('/'); });
};
```

- Senza la transition, fra `logout()` e `navigate('/')` React farebbe un render con `user === null` mentre siamo ancora su `/admin`: la pagina reindirizzerebbe a `/login`. Il commento nel codice lo dice.
- Altre novità di React 19 che incontreremo: `ref` come prop normale (niente `forwardRef`), `useActionState` per i form, `<title>` e `<meta>` scrivibili dentro i componenti.

Composizione: children e slot

- Non fate componenti con quindici prop booleane: passate **children**.
- È il pattern di shadcn/ui, che avete già nel progetto: `<Card>` contiene `<CardHeader>`, `<CardContent>`, `<CardFooter>`; voi decidete cosa ci va dentro.
- `asChild` di Radix: `<Button asChild><Link to="/">Catalogo</Link></Button>` dà l'aspetto del bottone a un link vero. Lo usa già `Header.tsx`.
- Custom hook: quando la stessa *logica* (non la stessa UI) serve in due posti, estraetela in `useQualcosa()`.

Esercizio in aula

`CatalogPage` è lunga più di 200 righe e fa tre cose. Dividetela in `ProductFilters`, `ProductGrid`, `ProductCard` e un hook `useProductFilters()`. È il refactoring più formativo del modulo.

Le route di oggi

```
// App.tsx, cosi' com'e': sei route, dentro <div className="min-h-screen">
<Header />
<Routes>
  <Route path="/" element={<CatalogPage />} />
  <Route path="/article/:articleid" element={<ArticlePage />} />
  <Route path="/cart" element={<CartPage />} />
  <Route path="/about" element={<AboutPage />} />
  <Route path="/login" element={<LoginPage />} />
  <Route path="/admin" element={<AdminPage />} />
</Routes>
<Toaster richColors position="top-right" />
```

- `<Header />` e `<Toaster />` stanno fuori da `<Routes>`: restano su tutte le pagine. È corretto.
- Manca una route `*`: un URL sbagliato mostra una pagina vuota con solo l'header.
- Le protezioni stanno dentro le pagine: `AdminPage` rimanda a `/login` chi non è amministratore, `CartPage` rimanda a `/admin` l'admin. Il controllo è già **duplicato**.

Route protette con un layout

```tsx
// components/ProtectedLayout.tsx -- con due ruoli serve anche isAdmin
export function ProtectedLayout({ adminOnly = false }: { adminOnly?: boolean }) {
  const { user } = useAuth();
  const location = useLocation();
  if (!user) return <Navigate to="/login" replace state={{ from: location }} />;
  if (adminOnly && !user.isAdmin) return <Navigate to="/" replace />;
  return <Outlet />;       // qui viene renderizzata la route figlia
}

// App.tsx
<Route element={<ProtectedLayout />}>
  <Route path="/cart" element={<CartPage />} />
</Route>
<Route element={<ProtectedLayout adminOnly />}>
  <Route path="/admin" element={<AdminPage />} />
</Route>
<Route path="*" element={<NotFoundPage />} />
```

Con `state: { from }` dopo il login si torna dove si voleva andare. Il campo `isAdmin` dell'utente divide clienti e amministratori: il carrello è dei clienti, l'area admin degli amministratori.

I filtri appartengono all'URL

```tsx
import { useSearchParams } from 'react-router-dom';

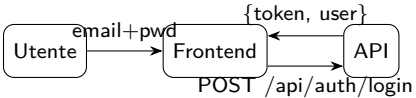
const [params, setParams] = useSearchParams();
const search = params.get('search') ?? '';
const sort   = (params.get('sort') ?? 'name') as SortField;

function setFilter(key: string, value: string) {
  setParams(prev => {
    if (value) prev.set(key, value); else prev.delete(key);
    return prev;
  }, { replace: true });    // replace: non riempie la cronologia a ogni tasto
}

<Input value={search} onChange={(e) => setFilter('search', e.target.value)} />
```

- Risultato: `/?search=mouse&sort=price` si può **inviare a un collega**, mettere nei preferiti, ricaricare.
- Il tasto Indietro torna ai filtri precedenti: è ciò che l'utente si aspetta.
- I parametri dell'URL hanno già la forma di quelli dell'API: meno conversioni.

Il flusso di login



1. Il backend confronta la password con `bcrypt` e firma un **JWT** che dura **8 ore**.
2. Il frontend salva token e utente in `localStorage` e li rimette allo **stato** React.
3. L'interceptor di axios allega il token a *ogni* richiesta successiva.
4. Al ricaricamento della pagina l'utente resta dentro, perché lo stato iniziale si legge da `localStorage`.
5. Stessa pagina, secondo percorso: `POST /api/auth/register` crea un **cliente** (`isAdmin: false`) e risponde con la stessa coppia `{token, user}`. Dopo l'accesso, l'amministratore va su `/admin`, il cliente sul catalogo.

L'AuthProvider del progetto

```tsx
export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<AuthUser | null>(loadStoredUser);   // lazy init

  const login = async (email: string, password: string) => {
    const { token, user } = await apiLogin(email, password);
    localStorage.setItem(TOKEN_KEY, token);
    localStorage.setItem(USER_KEY, JSON.stringify(user));
    setUser(user);
  };
  const logout = () => { /* rimuove le due chiavi e azzera lo stato */ };

  return <AuthContext value={{ user, login, logout }}>{children}</AuthContext>;
}
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth deve essere usato dentro AuthProvider');
  return ctx;
}
```

`useState(loadStoredUser)`: la funzione è passata *senza* parentesi, così viene eseguita una volta sola al primo render e non a ogni render.

Il buco: il token scaduto

Dopo 8 ore il token non vale più. Oggi l'utente vede una tabella vuota e un toast generico, ma resta "loggato".

```
// lib/api.ts: un interceptor di RISPOSTA accanto a quello di richiesta
api.interceptors.response.use((response) => response, (error) => {
  if (axios.isAxiosError(error) && error.response?.status === 401) {
    localStorage.removeItem(TOKEN_KEY);
    localStorage.removeItem(USER_KEY);
    window.dispatchEvent(new Event('auth:expired'));   // l'AuthProvider ascolta
  }
  return Promise.reject(error);
});

// AuthProvider: azzera l'utente quando arriva l'evento
useEffect(() => {
  const onExpired = () => setUser(null);
  window.addEventListener('auth:expired', onExpired);
  return () => window.removeEventListener('auth:expired', onExpired);
}, []);
```

Con la route protetta del layout, azzerare `user` basta: l'utente finisce su `/login` da solo.

Dove salvare il token

Opzione	Pro	Contro
Memoria (variabile JS)	immune al furto via storage	persa a ogni reload
`localStorage` (il progetto)	semplice, persiste	leggibile da qualsiasi script (XSS)
Cookie `httpOnly`	non leggibile da JS	va cambiato il backend, CSRF da gestire

- minicommerce usa `localStorage`: è anche la scelta che funziona identica in Electron, Tauri e Capacitor, dove i cookie sono più complicati.
- Nel modulo 6 vedremo perché per il *solo* web il cookie `httpOnly` è preferibile, e cosa cambierebbe nel backend.
- In ogni caso: il token è una **chiave**, non un segreto da nascondere nel codice. Chi ce l'ha, è l'utente.

Laboratorio 1a

Laboratorio: Routing e sessione (a mano, senza agenti)

1. Estrarre `ProtectedLayout` (con l'opzione `adminOnly`) e spostarci i controlli che oggi stanno dentro `AdminPage` e `CartPage`.
2. Aggiungere la route `*` con una pagina "404" che riporta al catalogo.
3. Interceptor di risposta per il 401 + evento `auth:expired` gestito dall'`AuthProvider`.
4. Dopo il login tornare alla pagina richiesta (`state.from`), non sempre a `/admin`.
5. Provare: cancellate a mano il token da DevTools, poi cliccate "Elimina" su un articolo.

Domande di verifica

Perché il controllo di autenticazione sta meglio nel layout? Cosa fa `replace` in `<Navigate>`?
Perché `useState(loadStoredUser)` e non `useState(loadStoredUser())`?

Il codice che c'è, e cosa gli manca

```
const [products, setProducts] = useState<Product[]>([]);
const [loading, setLoading] = useState(true);

useEffect(() => {
  const timer = setTimeout(() => {
    setLoading(true);
    fetchProducts({ search, category, minPrice, maxPrice, sort, order })
      .then(setProducts)
      .catch(err => toast.error(getErrorMessage(err)))
      .finally(() => setLoading(false));
  }, 250);
  return () => clearTimeout(timer);
}, [search, category, minPrice, maxPrice, sort, order]);
```

- Funziona. Ma: nessuno stato di **errore** in pagina, nessun **annullamento** della richiesta precedente, e la stessa logica è ricopiata in `AdminPage`, `ArticlePage`, `CartPage` e `ProductFormDialog`.
- Il difetto peggiore è il terzo: cinque copie che si comportano in modo leggermente diverso.

Un hook, una volta sola

```tsx
// lib/useProducts.ts
export function useProducts(filters: ProductFilters) {
  const [data, setData] = useState<Product[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [tick, setTick] = useState(0);            // reload(): fa ripartire l'effetto

  useEffect(() => {
    const ctrl = new AbortController();
    setLoading(true); setError(null);
    fetchProducts(filters, ctrl.signal)
      .then(setData)
      .catch(e => { if (!ctrl.signal.aborted) setError(getErrorMessage(e)); })
      .finally(() => { if (!ctrl.signal.aborted) setLoading(false); });
    return () => ctrl.abort();                    // annulla la richiesta precedente
  }, [JSON.stringify(filters), tick]);

  return { data, loading, error, reload: () => setTick(t => t + 1) };
}
```

Le due righe che contano

```ts
// lib/api.ts: basta accettare il signal e passarlo ad axios
export async function fetchProducts(filters: ProductFilters = {}, signal?: AbortSignal) {
  const { data } = await api.get<Product[]>('/products', { params: filters, signal });
  return data;
}
```

- `AbortController` risolve un bug che non avete ancora visto: scrivendo in fretta nella ricerca partono più richieste, e **non arrivano in ordine**. Senza annullamento, una risposta vecchia può sovrascrivere quella giusta.
- `JSON.stringify(filters)` nelle dipendenze è un trucco: l'oggetto dei filtri è nuovo a ogni render, quindi va confrontato per *valore*. L'alternativa pulita è costruirlo con `useMemo`.
- Lo stesso schema vale per `useCategories()`: due righe, nessun filtro.

La pagina diventa più corta

```
export function CatalogPage() {
  const [params] = useSearchParams();
  const filters = useDebounced(filtersFromParams(params), 250);
  const { data: products, loading, error, reload } = useProducts(filters);

  if (loading && products.length === 0) return <CatalogSkeleton />;
  if (error)   return <ErrorPanel message={error} onRetry={reload} />;
  if (products.length === 0) return <EmptyState />;

  return <ProductGrid products={products} busy={loading} />;
}
```

- La pagina non sa più *come* si caricano i dati: sa solo in quale stato si trova.
- `loading && products.length === 0`: scheletro solo al primo caricamento. Ai successivi restano i dati vecchi, leggermente sbiaditi.
- `AdminPage` usa lo stesso hook con filtri diversi, e la duplicazione sparisce.

Scrivere: creare, aggiornare, eliminare

```
// lib/useSaveProduct.ts
export function useSaveProduct(onSaved: () => void) {
  const [saving, setSaving] = useState(false);

  async function save(id: number | undefined, input: ProductInput) {
    setSaving(true);
    try {
      const saved = id ? await updateProduct(id, input) : await createProduct(input);
      toast.success(id ? 'Articolo aggiornato' : 'Articolo creato');
      onSaved();                          // chi possiede la lista la ricarica
      return saved;
    } finally {
      setSaving(false);                   // anche se l'API fallisce: il bottone si riattiva
    }
  }
  return { save, saving };
}
```

`AdminPage` possiede la lista e passa il suo `reload` come `onSaved`: è già quello che fa oggi il progetto, e a questa scala va benissimo.

Il carrello: lo stesso schema, su dati del server

```
// CartPage: il carrello non sta nel client, si legge e si riscrive
const loadCart = useCallback(() => {
  fetchCart().then(setCart).catch(e => toast.error(getErrorMessage(e)));
}, []);

async function cambiaQuantita(productId: number, quantity: number) {
  await updateCartItem(productId, quantity);   // PATCH /api/cart/items/:productId
  loadCart();                                  // il totale lo ricalcola il server
}
```

- Il carrello è di chi è loggato **come cliente**: a un token da amministratore l'API risponde 403.

L'ordine è finto

`placeMockOrder` chiama `POST /api/orders`: l'ordine è registrato con stato `accepted` e il carrello svuotato. Nessun pagamento, nessun controllo della carta, nessuno scarico delle scorte. Si salvano solo intestatario e ultime quattro cifre: numero completo, scadenza e CVV non vengono nemmeno inviati.

Aggiornamento ottimistico, a mano

```
function StockCell({ product, onChanged }: StockCellProps) {
  const [stock, setStock] = useState(product.stock);
  const [busy, setBusy] = useState(false);
  useEffect(() => setStock(product.stock), [product.stock]);   // la lista ricaricata vince

  async function cambia(delta: number) {
    const precedente = stock;
    setStock(stock + delta);                                   // 1. la UI cambia subito
    setBusy(true);
    try { await updateProduct(product.id, { ...toInput(product), stock: stock + delta });
     onChanged(); }
    catch (e) { setStock(precedente); toast.error(getErrorMessage(e)); }  // 2. si torna indietro
    finally { setBusy(false); }
  }
  ...
}
```

Nessuna libreria: due righe di stato locale. La UI risponde subito e, se il server rifiuta, il numero torna com'era. Non esiste una rotta per il solo stock: si riusa `updateProduct`, cioè `PUT /api/products/:id` con tutti i campi.

I quattro stati di ogni schermata

1. **Caricamento**: scheletro della griglia, non uno spinner al centro dello schermo.
2. **Errore**: messaggio comprensibile e pulsante *Riprova*. Oggi minicommerce mostra solo un toast che sparisce dopo 4 secondi.
3. **Vuoto**: c'è già, con icona e testo. Buon esempio da imitare.
4. **Dati**: la griglia.

Il quinto stato

Dati vecchi mentre ne arrivano di nuovi: con `busy` si abbassa un po' l'opacità della griglia. L'utente capisce che sta succedendo qualcosa senza perdere ciò che sta leggendo.

Regola

Un toast è un *avviso*, non uno stato. Se l'informazione serve ancora dopo cinque secondi, deve stare in pagina.

Quando il progetto cresce: le librerie

Quello che avete scritto sta in trenta righe e basta per minicommerce. Da una certa dimensione in poi servono cose che a mano costano care:

- **Cache condivisa** fra componenti e fra pagine: due schermate che chiedono le categorie fanno *una* richiesta.
- **Ricarica in background** quando si torna sulla scheda, e dopo un tempo configurabile.
- **Retry** automatici, deduplica delle richieste uguali, cache che sopravvive alla chiusura.

Da conoscere di nome

TanStack Query e **SWR** sono le due risposte standard nel mondo React, e le troverete in azienda. In questo corso non le usiamo: prima bisogna sapere *cosa* fanno al posto vostro, altrimenti diventano magia.

Laboratorio 1b

Laboratorio: Catalogo e admin su un hook solo

1. Spostare i filtri nell'URL con `useSearchParams`.
2. Scrivere `useProducts` e `useCategories` in `lib/`, con `AbortController` e `reload`; stessa forma per un `useCart()`.
3. `CatalogPage` e `AdminPage` usano gli hook: tutti e quattro gli stati presenti, `ErrorPanel` con Riprova.
4. `useSaveProduct` per creazione e modifica; l'eliminazione ricarica la lista.
5. Provare il bug delle risposte fuori ordine: rallentate `/api/products` con un `setTimeout` nel backend, digitate in fretta, e verificate che l'annullamento lo risolva.

Domande di verifica

Perché una risposta vecchia può sovrascrivere quella nuova? Cosa fa `AbortController`? Perché i filtri vanno confrontati per valore e non per riferimento? Chi possiede la lista degli articoli?

La validazione scritta a mano

```
// ProductFormDialog.tsx, com'e' oggi: 7 campi come stringhe + una funzione
function validateForm(form: typeof EMPTY_FORM): FieldErrors {
  const errors: FieldErrors = {};
  if (!form.name.trim()) errors.name = 'Il nome è obbligatorio.';
  else if (form.name.trim().length > 32) errors.name = 'Il nome non può superare 32 caratteri.';
  if (!form.categoryId) errors.categoryId = 'Seleziona una categoria.';
  if (form.price.trim() === '') errors.price = 'Il prezzo è obbligatorio.';
  else if (!Number.isFinite(Number(form.price)) || Number(form.price) < 0) { ... }
  // ... e cosi' via, piu' lo stato 'touched' per ogni campo
}
```

- Non è codice sbagliato: è codice **ripetitivo**, e ce ne sarà uno per ogni form che aggiungete.
- I valori sono tutti stringhe e vanno riconvertiti a mano in numeri prima dell'invio.
- Le regole stanno solo qui: non si possono riusare nei test né condividere con il backend.

Lo stesso form con zod

```
npm install react-hook-form zod @hookform/resolvers
```

```
export const productSchema = z.object({
  name: z.string().trim().min(1, 'Il nome è obbligatorio').max(32, 'Massimo 32 caratteri'),
  categoryId: z.coerce.number().int().positive('Seleziona una categoria'),
  brand: z.string().trim().max(50, 'Massimo 50 caratteri').default(''),
  description: z.string().trim().max(500, 'Massimo 500 caratteri').default(''),
  price: z.coerce.number().min(0, 'Il prezzo deve essere maggiore o uguale a 0'),
  stock: z.coerce.number().int().min(0, 'Deve essere un intero maggiore o uguale a 0').default(0),
  imageUrl: z.string().nullable().default(null),
});
export type ProductFormValues = z.infer<typeof productSchema>;   // il tipo nasce dallo schema
```

- `z.coerce` converte la stringa dell'`<input>` in numero *prima* di validare: sparisce la conversione manuale.
- Uno schema, tre usi: validazione del form, tipo TypeScript, e (se volete) controllo delle risposte dell'API.

react-hook-form nel dialog

```
const form = useForm<ProductFormValues>({
  resolver: zodResolver(productSchema),
  defaultValues: product ? toFormValues(product) : EMPTY_VALUES,
});
const { register, handleSubmit, reset, setError, formState: { errors, isSubmitting } } = form;

useEffect(() => { reset(product ? toFormValues(product) : EMPTY_VALUES); }, [product, reset]);

<form onSubmit={handleSubmit(onSubmit)} noValidate className="space-y-4">
  <Label htmlFor="p-name">Nome *</Label>
  <Input id="p-name" {...register('name')} aria-invalid={!!errors.name}
         aria-describedby={errors.name ? 'p-name-error' : undefined} />
  {errors.name && <p id="p-name-error" role="alert" className="text-sm text-destructive">
    {errors.name.message}</p>}
  <Button type="submit" disabled={isSubmitting}>Salva</Button>
</form>
```

Gli attributi di accessibilità (`aria-invalid`, `aria-describedby`, `role="alert"`) ci sono già nel progetto: li teniamo, cambia solo chi produce l'errore.

Gli errori del server

```
async function onSubmit(values: ProductFormValues) {
  try {
    await save(product?.id, values);        // il save di useSaveProduct
    onOpenChange(false);
  } catch (err) {
    // il backend risponde 400 { error: "Nessuna categoria con id 99" }
    if (axios.isAxiosError(err) && err.response?.status === 400) {
      const message = err.response.data?.error ?? 'Dati non validi';
      setError(message.toLowerCase().includes('categoria') ? 'categoryId' : 'root', { message });
    } else toast.error(getErrorMessage(err));
  }
}
```

- La validazione lato client migliora l'esperienza; quella lato server è l'unica che conta per la sicurezza. Entrambe servono.
- Il backend oggi restituisce un solo messaggio: se volete errori per campo, è una piccola modifica a `routes/products.ts`.

Il campo immagine: un caso da discutere

```
// Oggi: il file viene letto e trasformato in data URL base64, salvato nel DB
const reader = new FileReader();
reader.onload = () => setForm(c => ({ ...c, imageUrl: reader.result as string }));
reader.readAsDataURL(file);        // "..."
```

- Funziona senza toccare il backend, ed è il motivo per cui è stato fatto così.
- Però: un'immagine da 5 MB diventa circa **6,7 MB** di testo, entra nel database, e viaggia *dentro* ogni risposta di `/api/products`. Il catalogo con venti articoli diventa pesantissimo.
- La via corretta: `POST /api/products/:id/image` in `multipart/form-data`, il file su disco o su un bucket, nel database solo l'URL.
- Lo rifaremo nel modulo 3, quando la foto arriverà dalla fotocamera del telefono.

Che tipo di stato è?

Tipo	Esempi in minicommerce	Dove vive
Server state	articoli, categorie, **carrello**, ordine	hook + `lib/api.ts`
URL state	ricerca, categoria, prezzi, ordinamento	React Router
Form state	campi del dialog, errori	react-hook-form
UI locale	dialog aperto, articolo in modifica	`useState`
UI globale	tema chiaro/scuro, toast	Context o Zustand
Sessione	utente loggato e suo ruolo (`isAdmin`)	Context (`AuthProvider`)

L'errore più comune

Mettere i dati del server in uno store globale e reinventare a mano cache, invalidazione e sincronizzazione. Il carrello di minicommerce è il caso da ricordare: vive nel database, legato all'utente, e si legge con `GET /api/cart`.

Tema chiaro/scuro: il progetto è già pronto

```
// index.css del progetto: la variante dark c'e' gia'
@custom-variant dark (&:is(.dark *));
```

```
// next-themes e' gia' fra le dipendenze: manca solo usarlo
import { ThemeProvider, useTheme } from 'next-themes';

<ThemeProvider attribute="class" defaultTheme="system" enableSystem>
  <App />
</ThemeProvider>

function ThemeToggle() {
  const { theme, setTheme } = useTheme();
  return (
    <Button variant="ghost" size="icon" aria-label="Cambia tema"
            onClick={() => setTheme(theme === 'dark' ? 'light' : 'dark')}>
      {theme === 'dark' ? <Sun /> : <Moon />}
    </Button>);
}
```

`attribute="class"` mette `class="dark"` sulla radice: è ciò che la variante CSS si aspetta.

Context, Zustand, Redux

- **Context** (quello che usa il progetto): perfetto per valori che cambiano raramente, come l'utente loggato. Ogni consumatore si ri-renderizza a ogni cambiamento.
- **Zustand**: store minimale con selettori, utile per UI globale che cambia spesso. In minicommerce non serve: il candidato naturale sarebbe il carrello, ma il carrello **non è stato del client**.
- Vive sul server, legato all'utente: si legge con `fetchCart` e si scrive con `addCartItem`, `updateCartItem`, `removeCartItem`, `clearCart`. Uno store client sarebbe una seconda copia da tenere allineata.
- **Redux Toolkit**: ancora molto diffuso in azienda, utile per flussi complessi. Non lo useremo, ma dovete riconoscerlo.

Come fa l'header a sapere quanti articoli ci sono

Dopo ogni scrittura, `lib/api.ts` emette l'evento `minicommerce:cart-updated`; `Header.tsx` lo ascolta e richiama `fetchCart`. Nessuno store globale: una rilettura dalla fonte di verità, che è il server.

Il problema che Tailwind risolve

```
/* CSS "classico": tre domande a ogni componente nuovo */
.product-card__footer--highlighted { ... }   /* 1. come lo chiamo? */
.card .footer span { color: red; }           /* 2. chi vince? specificita' */
/* 3. questa regola serve ancora a qualcuno? (nessuno lo sa: resta li' per anni) */
```

```
<!-- Tailwind: le stesse decisioni, prese dove servono -->
<div className="flex items-center justify-between gap-2 border-t pt-3">
```

- Ogni classe fa **una** cosa sola e non ha specificità da contendere.
- Niente nomi da inventare, niente file da cercare, niente CSS morto: il build include solo le classi che compaiono davvero nel codice.
- Il prezzo: righe di markup lunghe. Si paga estraendo *componenti*, non scrivendo CSS.

Le classi che userete davvero

Gruppo	Esempi	A cosa serve
Layout	`flex`, `grid`, `grid-cols-3`, `hidden`	disporre gli elementi
Allineamento	`items-center`, `justify-between`	nel contenitore flex o grid
Spaziatura	`gap-4`, `p-4`, `px-3`, `mt-2`	scala a passi di 0,25 rem: $4 = 1$ rem
Dimensioni	`w-full`, `max-w-6xl`, `size-4`	`size-4` è larghezza + altezza
Tipografia	`text-sm`, `font-medium`, `truncate`	`line-clamp-3` taglia a tre righe
Colori	`bg-background`, `text-muted-foreground`	nomi *semantici*, non `gray-500`
Bordi	`rounded-lg`, `border`, `divide-y`	`divide-y` riga fra i figli

Sono le stesse che trovate in `CatalogPage`: `mx-auto max-w-6xl space-y-6 px-4 py-8`. Leggetele come una frase: centrata, larga al massimo 72 rem, con spazio fra i blocchi.

Prefissi: responsive, stati, tema

```
<!-- schermo: senza prefisso = telefono, poi si allarga -->
<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3">

<!-- stato dell'elemento -->
<button className="bg-primary hover:bg-primary/80 focus-visible:ring-3 disabled:opacity-50">

<!-- tema scuro (nel progetto: classe "dark" sulla radice) -->
<div className="bg-white dark:bg-zinc-900">

<!-- stato del genitore o di un fratello -->
<div className="group"> <span className="opacity-0 group-hover:opacity-100">Modifica</span> </div>
<input className="peer" /> <p className="hidden peer-invalid:block">Valore non valido</p>
```

- I prefissi si combinano: `lg:hover:bg-muted`, `dark:aria-invalid:border-destructive`.
- `bg-primary/80` è l'opacità dello sfondo: niente `rgba` da calcolare a mano.

Tailwind v4: la configurazione sta nel CSS

```
/* src/index.css del progetto, in ordine */
@import "tailwindcss";
@import "shadcn/tailwind.css";
@custom-variant dark (&:is(.dark *));   /* da qui nasce il prefisso dark: */

@theme inline {                          /* i nomi delle classi */
  --color-background: var(--background);
  --radius-lg: var(--radius);
}
:root {                                  /* i valori, in tema chiaro */
  --background: oklch(1 0 0);
  --destructive: oklch(0.577 0.245 27.325);
}
```

- Niente `tailwind.config.js`: in v4 il tema è CSS. `@theme` crea le classi, `:root` dà i valori.
- `-color-primary` genera `bg-primary`, `text-primary`, `border-primary`.
- Colori in **oklch**: cambiare `-primary` ricolora tutta l'applicazione.

Riuso: componenti, cn, varianti

```
// 1. cn(): unisce le classi e risolve i conflitti (l'ultima vince)
<div className={cn('rounded-lg border p-4', isLate && 'border-destructive', className)} />

// 2. cva(): un componente, piu' varianti. E' cosi' che e' fatto Button nel progetto
const buttonVariants = cva(
  'inline-flex items-center justify-center rounded-lg text-sm font-medium disabled:opacity-50',
  { variants: {
      variant: { default: 'bg-primary text-primary-foreground hover:bg-primary/80',
                 outline: 'border-border bg-background hover:bg-muted',
                 destructive: 'bg-destructive text-white' },
      size:    { default: 'h-9 px-4', sm: 'h-8 px-3', icon: 'size-9' } },
    defaultVariants: { variant: 'default', size: 'default' } },
);
<Button variant="destructive" size="icon" />
```

Senza `cn` una prop `className` passata da fuori finirebbe *accanto* a quella interna, e vincerebbe la più specifica a caso.

Tailwind: errori comuni

Da evitare

- Costruire nomi di classe a pezzi: `` `text-${colore}-500` ``. Il build non li trova e spariscono.
- Usare `gray-500` invece dei colori semantici: in tema scuro resta grigio sbagliato.
- Valori arbitrari ovunque (`w-[237px]`): se capita spesso, è un token che manca.
- Reinventare `@apply` per tornare al CSS di prima.

Da fare

- Estrarre un componente quando la stessa stringa compare tre volte.
- Installare l'estensione **Tailwind CSS IntelliSense**: completamento, anteprima del colore, ordinamento.
- Leggere `components/ui/`: è Tailwind scritto bene, ed è già nel vostro repository.
- Nel dubbio sul nome di una classe: `https://tailwindcss.com/docs`, la ricerca è ottima.

shadcn/ui: componenti copiati, non installati

```
npx shadcn@latest add checkbox pagination skeleton alert
# scrive i file in src/components/ui/: da quel momento sono VOSTRI
```

- Non è una libreria di componenti: è un **generatore**. Il codice finisce nel vostro repository e lo potete modificare.
- Sotto c'è **Radix UI**: dialog, select e menu accessibili (focus, tastiera, Esc, ARIA) già fatti bene.
- `components.json` dice dove mettere i file e quale stile usare: nel progetto, stile `radix-nova` e icone `lucide`.
- Vantaggio: zero attesa per una patch upstream. Svantaggio: gli aggiornamenti non arrivano da soli.

Da provare in laboratorio

Aggiungete `skeleton` e usatelo per lo stato di caricamento del catalogo, al posto della scritta "Caricamento...".

Il dialog: perché non va fatto a mano

```
<Dialog open={open} onOpenChange={onOpenChange}>
  <DialogContent className="sm:max-w-lg">
    <DialogHeader>
      <DialogTitle>{product ? 'Modifica articolo' : 'Nuovo articolo'}</DialogTitle>
      <DialogDescription>Compila i campi e salva.</DialogDescription>
    </DialogHeader>
    ...
  </DialogContent>
</Dialog>
```

- Radix vi dà gratis: focus spostato dentro all'apertura e restituito alla chiusura, chiusura con Esc, sfondo che blocca il resto, `aria-modal`, lettura del titolo da parte dello screen reader.
- Un `<div>` con `position: fixed` non fa nulla di tutto questo.
- Alternativa senza librerie: l'elemento HTML `<dialog>` con `showModal()`, che fa quasi altrettanto.

Da sostituire: window.confirm

```
// AdminPage oggi
const handleDelete = async (product: Product) => {
  if (!window.confirm(‘Eliminare "${product.name}" dal catalogo?‘)) return;
  ...
};
```

- `window.confirm` **blocca** il thread, non si può stilizzare, ha un aspetto diverso su ogni browser, e in Electron o in una webview mobile stona del tutto.
- Sostituzione: `AlertDialog` di shadcn/ui, con il nome dell'articolo nel testo e il pulsante distruttivo in rosso.
- Regola generale: `alert`, `confirm` e `prompt` non entrano in un'applicazione finita.

Laboratorio 1c

Laboratorio: Form, dialog, tema

1. Riscrivere `ProductFormDialog` con react-hook-form + zod, mantenendo tutti gli attributi ARIA esistenti.
2. Mappare l'errore 400 della categoria sul campo `categoryId`.
3. Sostituire `window.confirm` con `AlertDialog`; aggiungere `Skeleton` al caricamento del catalogo.
4. Attivare il tema chiaro/scuro con `next-themes` e un pulsante nell'header.

Domande di verifica

Perché lo schema zod genera anche il tipo? Cosa fa `z.coerce`? Quando un errore del server va su un campo e quando su `root`? Cosa vi dà Radix che un `<div>` non vi dà?

Debounce riutilizzabile

```ts
// hooks/useDebounced.ts  -- la stessa idea del timer in CatalogPage, ma riusabile
export function useDebounced<T>(value: T, ms = 250): T {
  const [v, setV] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setV(value), ms);
    return () => clearTimeout(id);
  }, [value, ms]);
  return v;
}

// uso: i filtri cambiano a ogni tasto, la richiesta parte dopo 250 ms di quiete
const filters = filtersFromParams(params);
const { data } = useProducts(useDebounced(filters));
```

- Senza debounce: una richiesta per ogni tasto premuto. Con 50 articoli non si nota; con 50.000 sì, e il server se ne accorge prima di voi.
- Attenzione a *cosa* si fa il debounce: l'input resta immediato, è la *query* a essere ritardata.

Paginazione: il catalogo non può crescere all'infinito

```
// Il backend oggi restituisce TUTTI gli articoli che passano i filtri.
// Primo passo (lato server): aggiungere limit e offset in routes/products.ts
//   .limit(Number(limit) || 24).offset(Number(offset) || 0)
// e restituire { items, total } invece di un array.

const [page, setPage] = useState(0);
const { data, loading } = useProducts({ ...filters, limit: 24, offset: page * 24 });
<Pagination page={page} total={data.total} pageSize={24} onChange={setPage} />

// la pagina cambia -> cambiano i filtri -> l'hook rifa' la richiesta, da solo
```

- In alternativa, **scorrimento infinito**: più adatto a un catalogo da telefono (modulo 3). Si accumulano le pagine invece di sostituirle.
- Per liste molto lunghe in area admin: **virtualizzazione** (per esempio TanStack Virtual), che disegna solo le righe visibili.

Modifica rapida della disponibilità

```tsx
// In tabella admin: +/- sullo stock senza aprire il dialog
function StockCell({ product, onChanged }: StockCellProps) {
  const { stock, busy, cambia } = useStockOttimistico(product, onChanged);
  return (
    <div className="flex items-center justify-end gap-1">
      <Button variant="ghost" size="icon" aria-label={`Togli una unità da ${product.name}`}
              disabled={busy || stock === 0} onClick={() => cambia(-1)}>
        <Minus className="size-3" />
      </Button>
      <span className="w-8 text-center tabular-nums">{stock}</span>
      <Button variant="ghost" size="icon" aria-label={`Aggiungi una unità a ${product.name}`}
              disabled={busy} onClick={() => cambia(+1)}>
        <Plus className="size-3" />
      </Button>
    </div>
  );
}
```

Con l'aggiornamento ottimistico il numero cambia istantaneamente: è la funzione che il commesso userà cento volte al giorno.

Error Boundary

```
npm install react-error-boundary
```

```
<ErrorBoundary
  FallbackComponent={({ error, resetErrorBoundary }) => (
    <section role="alert" className="mx-auto max-w-md py-16 text-center">
      <h2 className="text-lg font-semibold">Qualcosa è andato storto</h2>
      <p className="text-muted-foreground">{error.message}</p>
      <Button onClick={resetErrorBoundary}>Riprova</Button>
    </section>
  )}
>
  <Routes>...</Routes>
</ErrorBoundary>
```

- Senza boundary, un errore durante il rendering **smonta tutta l'app**: schermo bianco, header compreso.
- Provate: in `ProductCard` scrivete `product.category.toUpperCase()` e fate restituire al backend una categoria nulla.

Code splitting

```
import { lazy, Suspense } from 'react';
const AdminPage = lazy(() => import('@/pages/AdminPage'));

<Route element={<ProtectedLayout />}>
  <Route path="/admin" element={
    <Suspense fallback={<PageSkeleton />}><AdminPage /></Suspense>
  } />
</Route>
```

- Il cliente che apre il catalogo non ha motivo di scaricare l'area admin, il dialog, la tabella e la validazione.
- Vite crea un file JS per ogni `import()` dinamico. `npm run build` stampa le dimensioni di ciascuno.
- Regola: le pagine si dividono sempre; i componenti solo se pesano davvero.

Memoizzazione: quando serve davvero

- React ri-renderizza quando cambiano props o stato, o quando si ri-renderizza il padre. Nel 99% dei casi è **abbastanza veloce**.
- Misurate prima di ottimizzare: React DevTools, scheda Profiler, registrate mentre digitate nella ricerca.
- `useMemo` per calcoli costosi veri (raggruppare o ordinare migliaia di righe lato client).
- `useCallback` + `React.memo` quando passate funzioni a liste lunghe di figli memoizzati.
- **React Compiler**: memoizza da solo, si attiva con un plugin di Babel in `vite.config.ts`. Provatelo nel progetto finale e confrontate il profilo.

Il proxy non esiste in produzione

```
# .env.development
VITE_API_URL=/api                                   # il proxy di Vite fa il resto
# .env.production
VITE_API_URL=https://api.minicommerce.example/api
```

```
// lib/api.ts
export const api = axios.create({ baseURL: import.meta.env.VITE_API_URL ?? '/api' });
```

- Solo le variabili con prefisso `VITE_` finiscono nel bundle. Tutto il resto resta fuori: **nel client non ci sono segreti**.
- Con un'origine diversa dal backend servono le intestazioni **CORS**: il progetto ha già `app.use(cors())`, che in produzione va ristretto al vostro dominio (modulo 6).
- Questa stessa riga tornerà identica in Electron, Tauri e Capacitor: lì il frontend non è *mai* sulla stessa origine dell'API.

Build e deploy di una SPA

```
npm run build      # tsc -b && vite build  ->  dist/
npm run preview    # serve dist/ in locale, senza proxy: come in produzione
```

```
# nginx: qualsiasi percorso -> index.html, altrimenti /admin ricaricata da' 404
location / {
  try_files $uri $uri/ /index.html;
}
location /assets/ { add_header Cache-Control "public, max-age=31536000, immutable"; }
```

- `dist/` è una cartella statica: Netlify, Vercel, Cloudflare Pages, GitHub Pages o un'immagine `nginx:alpine` da 20 MB.
- Il backend va ospitato a parte (un piccolo server Node con il file SQLite accanto).
- Nel modulo 7 automatizziamo tutto con GitHub Actions.

Laboratorio 1d

Laboratorio: Rifinitura, build, pubblicazione

1. `useDebounced` riusabile; paginazione del catalogo (anche lato backend: `limit` e `offset`).
2. Pulsanti +/- sullo stock in area admin, con aggiornamento ottimistico.
3. Error boundary attorno alle route; `AdminPage` in lazy loading; controllare le dimensioni del bundle.
4. `VITE_API_URL`, `npm run build`, `npm run preview`; deploy del frontend su Netlify o Cloudflare Pages con la regola di fallback.

Domande di verifica

Perché `npm run preview` trova errori che `npm run dev` non mostra? Cosa succede aprendo direttamente `/admin` senza la regola di nginx? Cosa finisce nel bundle se scrivo `VITE_JWT_SECRET`?

Checklist: minicommerce Web è pronta?

- ☐ Login, logout, sessione persistente, 401 gestito
- ☐ Catalogo: filtri nell'URL, debounce, paginazione
- ☐ Admin: CRUD completo, stock rapido, conferme accessibili
- ☐ I quattro stati in ogni schermata
- ☐ Carrello e ordine finto provati da cliente
- ☐ Route protette per ruolo e pagina 404
- ☐ Tema chiaro/scuro
- ☐ Error boundary e code splitting
- ☐ Build pubblicata e raggiungibile
- ☐ README aggiornato con le vostre aggiunte
- ☐ Nessun segreto nel repository

Da qui in poi

Questa applicazione, *così com'è*, diventerà un programma per il desktop del negozio nel prossimo modulo. Tenete il codice pulito: lo riusiamo tutto.